

COURS DE JAVA

Cours complet — Fiche de révision (Bases + Programmation Orientée Objet)

Bases • Tableaux • Classes/objets • Chaînes • Héritage • Polymorphisme • Abstraction • Interfaces • Exceptions • Collections • Génériques • Enums • Classes internes

Préparé pour Maryame — ENS Marrakech

Sommaire

Chapitre 1 — Les bases de Java

1. C'est quoi Java, en une phrase ?

Java est un langage qui permet d'écrire un programme une seule fois et de le faire tourner sur n'importe quel ordinateur (Windows, Mac, Linux...), grâce à une machine virtuelle appelée la JVM. C'est le fameux principe « Write Once, Run Anywhere ».

Pour bien comprendre le mécanisme, imagine que tu écris une recette de cuisine (ton code). La JVM est comme un traducteur universel qui comprend cette recette et sait la refaire quel que soit le pays (le système d'exploitation) où tu te trouves.

2. JDK et JRE : ne plus jamais les confondre

C'est LA question de cours classique. Retiens l'analogie suivante :

- JRE (Java Runtime Environment) = la cuisine déjà équipée. Il contient la JVM et les bibliothèques nécessaires pour exécuter un programme déjà écrit. Suffisant si tu veux seulement lancer un logiciel Java.
- JDK (Java Development Kit) = la cuisine + le livre de recettes + les outils du chef (couteaux, etc.). Il contient le JRE + le compilateur javac + javadoc + jar + jdb. Nécessaire si tu veux écrire et compiler du code.

Moyen simple de retenir : JDK = JRE + outils de développement. Si tu développes, il te faut le JDK.

Version	Sortie	Support Oracle
Java 8 (LTS)	mars 2014	déc. 2022
Java 11 (LTS)	sept. 2018	sept. 2023
Java 17 (LTS)	sept. 2021	sept. 2024
Java 21 (LTS)	sept. 2023	sept. 2026

Astuce à retenir : les versions « LTS » (Long Term Support) sortent tous les 2-3 ans et sont celles qu'on utilise en entreprise. Les autres versions (9, 10, 12...) sortent tous les 6 mois mais ne sont pas maintenues longtemps.

3. Compiler puis exécuter : les deux étapes obligatoires

Un programme Java ne s'exécute jamais directement. Il passe toujours par deux étapes :

- 1) La compilation : javac transforme ton fichier .java (texte lisible) en fichier .class (bytecode, illisible pour un humain).
- 2) L'exécution : la commande java lance la JVM, qui lit le bytecode et l'exécute réellement.

```
javac Bonjour.java      // étape 1 : crée Bonjour.class
java Bonjour            // étape 2 : exécute (SANS l'extension .class)
```

Piège classique : au moment de l'exécution, on écrit java Bonjour et non java Bonjour.class — c'est une erreur très fréquente en TP.

4. La structure minimale d'un programme

Tout programme Java a besoin d'une classe et d'une méthode main, qui est le point de départ de l'exécution :

```
public class Bonjour {
    public static void main(String[] args) {
        System.out.println("Bienvenue dans le monde Java !");
    }
}
```

```
}  
}
```

Règle absolue : le nom du fichier doit être identique au nom de la classe publique. Ici, le fichier doit s'appeler Bonjour.java (sinon erreur de compilation).

5. Les types primitifs (les briques de base)

Avant d'utiliser une variable, il faut la déclarer avec un type. Les 4 types les plus utilisés :

Type	Contient	Exemple
int	un nombre entier	int age = 25;
double	un nombre à virgule	double prix = 19.99;
boolean	vrai ou faux	boolean estValide = true;
char	un seul caractère	char lettre = 'M';

Astuce examen : « double » utilise un point (pas une virgule) pour les décimales : 36.5, jamais 36,5.

6. Les opérations arithmétiques

Les opérateurs de base sont + - * / (division) et % (reste de la division, appelé modulo).

```
int a = 10, b = 3;  
System.out.println(a + b); // 13  
System.out.println(a * b); // 30  
System.out.println(a / b); // 3 (division ENTIÈRE : la partie décimale est coupée)  
System.out.println(a % b); // 1 (reste de 10 divisé par 3)
```

Piège n°1 à l'examen : a / b entre deux int donne toujours un résultat entier, même si le vrai résultat a une virgule. 10 / 3 vaut 3, pas 3.33. Pour avoir 3.33, il faut qu'au moins un des deux nombres soit un double.

7. Les conditions : faire des choix

Le programme choisit un chemin selon qu'une condition est vraie ou fausse.

if / else — l'exemple classique de la moyenne

```
if (note >= 10) {  
    System.out.println("Admis");  
} else {  
    System.out.println("Ajourné");  
}
```

if / else if / else — plusieurs niveaux

```
if (score >= 90) {  
    System.out.println("Excellent");  
} else if (score >= 75) {  
    System.out.println("Très bien");  
} else {  
    System.out.println("Peut mieux faire");  
}
```

switch — pratique quand on compare une seule variable à plusieurs valeurs

```
switch (jour) {  
    case 1 -> System.out.println("Lundi");  
    case 2 -> System.out.println("Mardi");  
}
```

```
default -> System.out.println("Jour inconnu");
}
```

8. Les boucles : répéter des actions

Boucle	Quand l'utiliser
for	Quand on connaît le nombre de répétitions à l'avance
while	Quand on répète tant qu'une condition est vraie (nombre inconnu à l'avance)
do...while	Comme while, mais le bloc s'exécute au moins une fois avant de tester la condition

```
for (int i = 1; i <= 5; i++) {           // affiche 1,2,3,4,5
    System.out.println("Compteur : " + i);
}

int i = 1;
while (i <= 3) {                        // s'arrête AVANT d'exécuter si i > 3
    System.out.println("i = " + i);
    i++;
}

int j = 1;
do {                                    // exécute au moins 1 fois, même si la
    condition est fausse
    System.out.println("j = " + j);
    j++;
} while (j <= 3);
```

9. Lire ce que tape l'utilisateur : Scanner

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Entrez un âge : ");
int age = sc.nextInt();                // lit un entier

System.out.print("Votre prénom : ");
String nom = sc.nextLine();           // lit une ligne de texte
```

Piège très classique : si on appelle `nextInt()` puis `nextLine()` juste après, `nextLine()` récupère souvent une chaîne vide, car `nextInt()` ne « consomme » pas le retour à la ligne tapé par l'utilisateur. Solution : ajouter un `sc.nextLine()` supplémentaire pour « vider » ce retour à la ligne avant de lire du texte.

10. Les méthodes : découper son code

Une méthode est un bloc de code réutilisable. Elle a un nom, des paramètres (optionnels) et un type de retour (ou void si elle ne renvoie rien).

```
public static int addition(int a, int b) {
    return a + b;
}

public static void main(String[] args) {
    int res = addition(4, 7); // res vaut 11
    System.out.println("Résultat : " + res);
}
```

Varargs : quand on ne connaît pas le nombre d'arguments

```
public static int somme(int... nombres) {    // "... " = nombre variable d'arguments
    int total = 0;
    for (int n : nombres) total += n;
    return total;
}
// Appels possibles : somme(5,10,20) ou somme(1) ou somme() -> tous valides
```

✓ Points clés du chapitre 1

- JDK = JRE + outils de développement (javac, jar, javadoc...).
- Deux étapes obligatoires : javac (compiler) puis java (exécuter, sans .class).
- Le nom du fichier .java doit correspondre EXACTEMENT au nom de la classe publique.
- int / int = division entière (tronquée) ; utiliser un double pour garder les décimales.
- for = nombre de répétitions connu ; while = condition testée avant ; do...while = testée après (exécute au moins 1 fois).
- Scanner : attention à l'enchaînement nextInt() puis nextLine().

⚠ Pièges fréquents à l'examen

- Écrire java Bonjour.class au lieu de java Bonjour.
- Oublier que $7 / 2$ vaut 3 (pas 3.5) car les deux opérandes sont des int.
- Confondre = (affectation) et == (comparaison).
- Oublier les accolades { } d'un bloc if à plusieurs lignes.
- Nom de fichier différent du nom de la classe publique.

🧠 Mini-quiz de révision — Chapitre 1

1. Que contient le JDK que le JRE n'a pas ?

→ Les outils de développement : javac, javadoc, jar, jdb.

2. Que vaut $17 \% 5$?

→ 2 (le reste de 17 divisé par 5).

3. Quelle boucle exécute son bloc au moins une fois, même si la condition est fausse dès le départ ?

→ do...while

4. Quelle commande sert à compiler un fichier Java ?

→ javac NomFichier.java

Chapitre 2 — Les tableaux (arrays)

1. Pourquoi utiliser un tableau ?

Un tableau permet de stocker plusieurs valeurs du MÊME type dans une seule variable, plutôt que de créer `note1`, `note2`, `note3`... Imagine un casier à bouteilles : une seule structure, plusieurs cases numérotées à partir de 0.

2. Tableau à une dimension

```
int[] notes = new int[5];          // 5 cases, toutes à 0 par défaut
int[] notes2 = {12, 14, 16, 10, 8}; // initialisation directe

System.out.println(notes2[2]);     // affiche 16 (indice 2 = 3e élément)
notes2[2] = 18;                   // on modifie la 3e case
```

Valeurs par défaut si on ne remplit pas le tableau : 0 pour les nombres, `false` pour boolean, `'\u0000'` (caractère vide) pour char.

Parcourir un tableau

```
// Boucle for classique (utile si on a besoin de l'indice i)
for (int i = 0; i < notes2.length; i++) {
    System.out.println("Note " + i + " = " + notes2[i]);
}

// Boucle for-each (plus simple, mais juste en LECTURE)
for (int note : notes2) {
    System.out.println(note);
}
```

Piège n°1 : `length` est un attribut (sans parenthèses) pour un tableau : `notes2.length`. Pour une String, c'est une méthode : `texte.length()`. Confondre les deux est une erreur très fréquente.

3. Tableaux à deux dimensions (matrices)

Utile pour représenter un plateau de jeu, une grille, ou un tableau de notes par élève et par matière.

```
int[][] matrice = new int[3][4]; // 3 lignes, 4 colonnes
int[][] identite = {
    {1, 0, 0},
    {0, 1, 0},
    {0, 0, 1}
};

for (int i = 0; i < matrice.length; i++) { // parcourt les lignes
    for (int j = 0; j < matrice[i].length; j++) { // parcourt les colonnes de la
        ligne i
        System.out.print(matrice[i][j] + " ");
    }
    System.out.println();
}
```

Un tableau « irrégulier » (ragged array) est un tableau 2D où chaque ligne peut avoir une longueur différente, par exemple pour représenter un triangle :

```
int[][] triangle = new int[4][]; // 4 lignes, longueur de colonne libre
triangle[0] = new int[1];
triangle[1] = new int[2];
triangle[2] = new int[3];
triangle[3] = new int[4];
```

4. Passer un tableau à une méthode

```
public static int somme(int[] t) {
    int s = 0;
    for (int n : t) s += n;
    return s;
}
```

Point important à retenir pour l'examen : en Java, un tableau est un objet. Quand on le passe à une méthode, on transmet une référence vers le même tableau en mémoire. Donc si la méthode modifie une case du tableau, la modification est visible aussi en dehors de la méthode.

5. Trier un tableau

Le tri intégré (le plus rapide et le plus utilisé en pratique) :

```
import java.util.Arrays;
int[] chiffres = {5, 2, 9, 1, 4};
Arrays.sort(chiffres); // chiffres devient [1, 2, 4, 5, 9]
```

Les tris « manuels » sont surtout demandés à l'examen pour vérifier qu'on comprend la logique algorithmique :

Tri à bulles (bubble sort) — compare les voisins et les échange

```
public static void bubbleSort(int[] t) {
    for (int pass = 1; pass < t.length; pass++) {
        for (int i = 0; i < t.length - pass; i++) {
            if (t[i] > t[i + 1]) {
                int tmp = t[i];
                t[i] = t[i + 1];
                t[i + 1] = tmp;
            }
        }
    }
}
```

Tri par sélection — cherche le minimum et le place au bon endroit

```
public static void selectionSort(int[] t) {
    for (int i = 0; i < t.length - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < t.length; j++) {
            if (t[j] < t[minIdx]) minIdx = j;
        }
        int tmp = t[i]; t[i] = t[minIdx]; t[minIdx] = tmp;
    }
}
```

Moyen de les différencier à l'examen : le tri à bulles compare des voisins immédiats et les échange tout de suite ; le tri par sélection cherche d'abord le minimum de toute la zone restante, puis fait UN SEUL échange par tour.

6. Filtrer un tableau (garder certains éléments)

La méthode à retenir en 3 étapes : 1) compter combien d'éléments respectent la condition, 2) créer un nouveau tableau de la bonne taille, 3) le remplir.

```
public static int[] filterEven(int[] t) {
    int count = 0;
    for (int n : t) if (n % 2 == 0) count++; // 1. compter

    int[] evens = new int[count]; // 2. allouer

    int idx = 0;
    for (int n : t) { // 3. remplir
        if (n % 2 == 0) evens[idx++] = n;
    }
}
```

```
}  
return evens;  
}
```

✓ Points clés du chapitre 2

- Les indices commencent à 0 et vont jusqu'à `length - 1`.
- Un tableau a une TAILLE FIXE dès sa création : impossible de l'agrandir.
- `tableau.length` (sans parenthèses) — à ne pas confondre avec `chaine.length()`.
- Un tableau passé à une méthode est modifié « pour de vrai » (c'est une référence).
- Filtrage = toujours compter avant d'allouer, pour ne pas avoir de cases inutilisées.

⚠ Pièges fréquents à l'examen

- Utiliser `i <= t.length` dans une boucle → provoque une `ArrayIndexOutOfBoundsException` (le bon indice max est `length - 1`).
- Écrire `tableau.length()` avec des parenthèses.
- Croire qu'on peut « ajouter » une case à un tableau déjà créé.
- Confondre tri à bulles et tri par sélection dans une explication écrite.

Mini-quiz de révision — Chapitre 2

1. Quel est le dernier indice valide d'un tableau `int[] t = new int[8]` ?

→ 7 (les indices vont de 0 à `length - 1`).

2. Quelle exception est levée si on accède à un indice trop grand ?

→ `ArrayIndexOutOfBoundsException`

3. Quelle est la différence principale entre tri à bulles et tri par sélection ?

→ Le tri à bulles échange des voisins à chaque comparaison ; le tri par sélection cherche le minimum puis fait un seul échange par tour.

Chapitre 3 — Classes et objets (POO)

1. L'idée de base de la POO

La Programmation Orientée Objet organise le code autour d'« objets » qui représentent des choses réelles (une personne, une voiture, un compte bancaire), plutôt qu'autour de simples fonctions.

L'analogie à retenir absolument pour l'examen :

- Une classe = le plan d'architecte d'une maison (le modèle).
- Un objet = une maison réellement construite à partir de ce plan (une instance).
- On peut construire plusieurs maisons (objets) à partir du même plan (classe), chacune avec sa propre couleur, ses propres meubles (son propre état).

2. Écrire une classe

```
public class Personne {  
    // Les attributs et méthodes viennent ici  
}
```

Règle stricte : le nom du fichier (Personne.java) doit correspondre exactement au nom de la classe publique.

3. Attributs = l'état de l'objet

```
public class Personne {  
    private String nom;    // chaque objet a SON PROPRE nom  
    private int age;       // chaque objet a SON PROPRE âge  
}
```

Si on crée alice et bob à partir de la classe Personne, modifier alice.age ne changera jamais bob.age : ce sont deux « maisons » différentes construites à partir du même plan.

4. Méthodes = les actions possibles

```
public void sePresenter() {  
    System.out.println("Je m'appelle " + nom + ", j'ai " + age + " ans.");  
}  
  
public int getAge() {  
    return age;  
}
```

void = la méthode ne renvoie rien. Si elle renvoie une valeur, il faut préciser le type de retour et utiliser return.

5. Le constructeur : donner vie à l'objet

Le constructeur porte le même nom que la classe et sert à initialiser les attributs dès la création de l'objet.

```
public Personne(String nom, int age) {  
    this.nom = nom;    // this.nom = l'attribut de l'objet  
    this.age = age;    // age (à droite) = le paramètre reçu  
}
```

- Si on n'écrit aucun constructeur, Java en crée un par défaut, sans paramètre.
- On peut écrire plusieurs constructeurs (« surcharge ») pour proposer différentes façons de créer un objet.

```
public Personne() {  
    this("Inconnu", 0);    // appelle l'autre constructeur avec des valeurs par défaut  
}
```

Créer un objet avec new

```
Personne p = new Personne("Alice", 30);
```

```
p.sePresenter();
System.out.println("Âge : " + p.getAge());
```

6. Encapsulation : protéger les données

L'encapsulation, c'est cacher les attributs (private) et ne laisser accéder à leur valeur que via des méthodes contrôlées (get/set). Cela évite qu'on mette n'importe quelle valeur, y compris invalide.

```
public void setAge(int age) {
    if (age >= 0) this.age = age;
    else throw new IllegalArgumentException("Âge invalide !");
}
```

Modificateur	Qui peut y accéder ?
private	Uniquement le code à l'intérieur de la même classe
(rien, par défaut)	Le code du même package
protected	Le même package + les sous-classes (héritage)
public	N'importe quel code, partout

Bonne pratique universelle : attributs toujours private, accès via des méthodes get/set publiques.

7. Le mot-clé this

this désigne toujours « l'objet en train d'exécuter le code ». Deux usages principaux :

- Distinguer un attribut d'un paramètre qui porte le même nom : `this.nom = nom;`
- Appeler un autre constructeur de la même classe : `this("Inconnu", 0);`

8. Les membres static : partagés par tous les objets

Un attribut ou une méthode static appartient à la CLASSE elle-même, pas à un objet particulier. Toutes les instances partagent la même valeur.

```
public class Compteur {
    private static int totalInstances = 0;    // UNE seule valeur, partagée

    public Compteur() {
        totalInstances++;    // chaque nouvel objet incrémente le compteur commun
    }

    public static int getTotalInstances() {
        return totalInstances;
    }
}
// Utilisation : Compteur.getTotalInstances(); -> on appelle via la CLASSE, pas un objet
```

Astuce pour retenir : static = « écrit sur le tableau de la classe entière », alors qu'un attribut normal est « écrit sur la porte de chaque maison individuelle ».

9. Personnaliser toString() et equals()

```
@Override
public String toString() {
    return "Personne[nom=" + nom + ", age=" + age + "]";
}
```

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Personne)) return false;
    Personne other = (Personne) obj;
    return this.nom.equals(other.nom) && this.age == other.age;
}
```

toString() sert à afficher un objet de façon lisible (au lieu d'une adresse mémoire illisible du type Personne@1b6d3586). equals() compare le CONTENU logique de deux objets, alors que == compare si ce sont exactement le même objet en mémoire.

✓ Points clés du chapitre 3

- Classe = plan/modèle ; Objet = instance concrète créée avec new.
- Chaque objet a sa propre copie des attributs (sauf s'ils sont static).
- Le constructeur porte le même nom que la classe et initialise les attributs.
- this = l'objet courant ; sert à lever une ambiguïté ou appeler un autre constructeur.
- Encapsulation = attributs private + méthodes get/set publiques.
- static = appartient à la classe entière, partagé par tous les objets (pas besoin d'un objet pour y accéder).
- equals() compare le contenu ; == compare l'adresse mémoire (donc si c'est le même objet).

⚠ Pièges fréquents à l'examen

- Oublier this. et écrire nom = nom; (l'attribut ne reçoit jamais la valeur du paramètre).
- Confondre == et equals() pour comparer deux objets.
- Appeler une méthode static via un objet (ex: p.getTotalInstances()) au lieu de la classe (Compteur.getTotalInstances()) — ça marche mais c'est une mauvaise pratique signalée à l'examen.
- Rendre les attributs public au lieu de private (rompt l'encapsulation).

🧠 Mini-quiz de révision — Chapitre 3

1. Quelle est la différence entre une classe et un objet ?

→ La classe est le modèle/plan ; l'objet est une instance concrète créée à partir de ce modèle.

2. Pourquoi écrire this.nom = nom; et pas juste nom = nom; ?

→ Sans this, Java affecte le paramètre à lui-même : l'attribut de l'objet n'est jamais mis à jour.

3. Un attribut static est-il propre à chaque objet ou partagé ?

→ Il est partagé par toutes les instances de la classe.

4. Que compare exactement equals() dans l'exemple Personne ?

→ Le nom et l'âge (le contenu logique), pas l'adresse mémoire.

Chapitre 4 — Les chaînes de caractères

1. Trois classes, trois usages différents

Classe	Rôle	Quand l'utiliser
String	Chaîne immuable (ne change jamais après création)	Le cas le plus courant, 95% du temps
StringBuffer	Chaîne mutable et thread-safe (synchronisée)	Beaucoup de modifications, notamment en environnement multithread
StringTokenizer	Découpe une chaîne en mots (tokens)	Découpage simple ; sinon préférer split() ou Scanner

2. String : l'essentiel

2.1 L'immuabilité : le concept clé du chapitre

Une fois qu'un String est créé, il ne peut JAMAIS être modifié. Toute opération qui « semble » le modifier (toUpperCase, concat, replace...) crée en réalité un NOUVEL objet String en mémoire, et laisse l'ancien inchangé.

```
String s1 = "Bonjour";
s1.toUpperCase();      // NE MODIFIE PAS s1 !
System.out.println(s1); // affiche toujours "Bonjour"

s1 = s1.toUpperCase(); // il faut RÉ-AFFECTER le résultat
System.out.println(s1); // affiche maintenant "BONJOUR"
```

C'est le piège numéro 1 de tout le chapitre : oublier de récupérer (réaffecter) le résultat d'une méthode de String.

2.2 Créer une String

```
String s1 = "Bonjour"; // littéral, stocké dans un pool partagé (économe en mémoire)
String s2 = new String("Java"); // force la création d'un nouvel objet en mémoire
char[] lettres = {'t','e','x','t','e'};
String s3 = new String(lettres); // construite depuis un tableau de caractères
```

2.3 Les méthodes utiles au quotidien

```
String s = "Bonjour Java";

s.length();           // 12 -> nombre de caractères
s.substring(8);        // "Java" (à partir de l'indice 8 jusqu'à la fin)
s.substring(0, 7);     // "Bonjour" (de l'indice 0 inclus à 7 EXCLU)
s.contains("Java");    // true
s.indexOf("o");        // 1 (position de la 1ère occurrence de 'o')
s.equals("Bonjour Java"); // true -> compare le CONTENU
s.equalsIgnoreCase("BONJOUR JAVA"); // true -> ignore majuscules/minuscules
s.toUpperCase();       // "BONJOUR JAVA"
s.trim();              // enlève les espaces au début et à la fin
```

Piège n°2 très classique : utiliser == pour comparer deux String au lieu de .equals(). == compare si c'est le même objet en mémoire, alors que .equals() compare le contenu (le texte). Avec des String créées via new String(...), == peut renvoyer false même si le texte est identique !

```
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b); // false (deux objets différents en mémoire)
```

```
System.out.println(a.equals(b)); // true (même contenu texte)
```

3. StringBuffer : quand on modifie beaucoup

Contrairement à String, un StringBuffer peut être modifié directement, sans créer de nouvel objet à chaque fois. Très utile dans une boucle avec beaucoup de concaténations (ce qui serait lent et coûteux en mémoire avec des String).

```
StringBuffer sb = new StringBuffer("Java");

sb.append(" Buffer"); // sb devient "Java Buffer"
sb.insert(4, " 2026"); // insère à l'indice 4
sb.delete(4, 9); // supprime les caractères entre les indices 4 et 8
sb.replace(0, 4, "JDK"); // remplace "Java" par "JDK"
sb.reverse(); // inverse tous les caractères
```

```
String resultat = sb.toString(); // convertit en String classique à la fin
```

Remarque pour l'examen : toutes les méthodes de StringBuffer sont synchronisées (sécurisées si plusieurs threads y accèdent en même temps), ce qui la rend un peu plus lente que sa cousine StringBuilder (identique mais NON synchronisée, à utiliser si on n'a qu'un seul thread).

4. StringTokenizer : découper une phrase en mots

```
String phrase = "Java;Python,C++;JavaScript";
StringTokenizer st = new StringTokenizer(phrase, ";,"); // délimiteurs personnalisés
```

```
while (st.hasMoreTokens()) {
    String mot = st.nextToken();
    System.out.println(mot);
}
// Affiche : Java Python C++ JavaScript
```

Sans délimiteur précisé, StringTokenizer découpe par défaut sur les espaces, tabulations et retours à la ligne :

```
StringTokenizer st2 = new StringTokenizer("un deux trois");
// tokens produits : "un", "deux", "trois"
```

À savoir pour la culture générale (souvent une question de cours) : StringTokenizer est une classe ancienne, aujourd'hui on lui préfère souvent String.split("\\s+") ou Scanner, mais elle reste simple et parfois demandée en TP.

5. Exemple récapitulatif (mélange des trois classes)

```
Scanner scanner = new Scanner(System.in);
System.out.print("Saisir une phrase : ");
String ligne = scanner.nextLine();

// 1. Comptage de mots avec split()
String[] mots = ligne.trim().split("\\s+");
System.out.println("Nombre de mots : " + mots.length);

// 2. Inversion avec StringBuffer (efficace, pas de String intermédiaires)
StringBuffer sb = new StringBuffer(ligne);
String inverse = sb.reverse().toString();
System.out.println("Phrase inversée : " + inverse);

// 3. Tokenisation avec StringTokenizer
StringTokenizer st = new StringTokenizer(ligne);
while (st.hasMoreTokens()) {
    String mot = st.nextToken();
    System.out.println(" • \"" + mot + "\"" (longueur : " + mot.length() + ")");
}
```

✓ Points clés du chapitre 4

- String est immuable : chaque « modification » crée un nouvel objet — il faut réaffecter le résultat.
- Toujours comparer le contenu de deux String avec `.equals()`, jamais avec `==` (qui compare la mémoire).
- `StringBuffer` est mutable et synchronisé : utile pour beaucoup de modifications ou en multithread.
- `StringBuilder` = comme `StringBuffer` mais non synchronisé, plus rapide en mono-thread.
- `StringTokenizer` découpe une chaîne en mots selon des délimiteurs (par défaut : espaces).

⚠ Pièges fréquents à l'examen

- Écrire `s.toUpperCase()`; seul, sans faire `s = s.toUpperCase()`, et croire que `s` a changé.
- Comparer deux String avec `==` au lieu de `.equals()`.
- Confondre `substring(0,7)` : la borne de fin (7) est EXCLUE, pas incluse.
- Oublier que `.length()` sur une String a des parenthèses (contrairement à `.length` sur un tableau).

🧠 Mini-quiz de révision — Chapitre 4

1. Pourquoi dit-on que String est « immuable » ?

→ Parce qu'un objet String, une fois créé, ne peut plus être modifié ; toute opération renvoie un nouvel objet.

2. Quelle est la bonne façon de comparer le contenu de deux String ?

→ Avec la méthode `.equals()`, pas avec `==`.

3. Quelle classe choisir pour de nombreuses modifications de texte dans une boucle ?

→ `StringBuffer` (ou `StringBuilder` en mono-thread).

4. Que renvoie `"Bonjour".substring(0,3)` ?

→ `"Bon"` (de l'indice 0 inclus à 3 exclu).

Chapitre 5 — L'héritage

1. L'idée : réutiliser ce qui existe déjà

L'héritage permet à une classe (la « sous-classe » ou « classe fille ») de récupérer automatiquement les attributs et méthodes d'une autre classe (la « super-classe » ou « classe mère »), puis d'ajouter ou de modifier ce dont elle a besoin en plus.

Analogie à retenir : une classe fille EST une classe mère « + quelque chose ». Un Etudiant EST une Personne, mais avec en plus un numéro d'étudiant. On dit qu'il existe une relation « est-un » (is-a).

```
public class Personne {
    protected String nom;
    public Personne(String nom) { this.nom = nom; }
    public void sePresenter() {
        System.out.println("Je m'appelle " + nom);
    }
}

public class Etudiant extends Personne {    // extends = "hérite de"
    private String numero;

    public Etudiant(String nom, String numero) {
        super(nom);        // appelle le constructeur de Personne
        this.numero = numero;
    }
}
```

Le mot-clé `extends` déclare l'héritage. `super(...)` appelle le constructeur de la classe mère : il doit obligatoirement être la première instruction du constructeur de la classe fille.

2. Le mot-clé `super`

`super` a deux usages, à ne pas confondre avec `this` :

- `super(...)` : appeler le constructeur de la classe mère.
- `super.méthode()` ou `super.attribut` : accéder explicitement à une méthode ou un attribut de la classe mère (utile si la classe fille l'a redéfini).

```
public class Etudiant extends Personne {
    @Override
    public void sePresenter() {
        super.sePresenter();    // exécute d'abord la version de Personne
        System.out.println("Je suis étudiant, numéro : " + numero);
    }
}
```

3. La redéfinition de méthode (`@Override`)

Une classe fille peut redéfinir (« override ») une méthode héritée pour lui donner un comportement différent, en gardant exactement la même signature (même nom, mêmes paramètres).

Piège classique : `@Override` n'est pas obligatoire pour que ça fonctionne, mais c'est fortement recommandé — le compilateur vérifie alors qu'on redéfinit bien une méthode existante, et signale une erreur si on a fait une faute de frappe dans le nom.

4. Simple héritage uniquement

En Java, une classe ne peut hériter que d'UNE seule classe (`extends UneSeuleClasse`). Contrairement à d'autres langages, l'héritage multiple de classes n'existe pas — c'est pour cela que les interfaces existent (voir chapitre 8), car une classe peut implémenter PLUSIEURS interfaces.

Modificateur protected

protected rend un attribut ou une méthode accessible depuis le même package ET depuis les classes filles, même dans un autre package. C'est le niveau de visibilité typique pour ce qu'une classe mère veut partager avec ses classes filles sans le rendre public à tout le monde.

✓ Points clés du chapitre 5

- extends = héritage (relation "est-un") ; une classe ne peut hériter que d'une seule classe.
- super(...) appelle le constructeur de la classe mère, en première ligne du constructeur fille.
- super.methode() appelle explicitement la version de la classe mère.
- @Override signale qu'on redéfinit une méthode héritée — le compilateur vérifie la signature.
- protected = visible dans le même package + dans les classes filles (même ailleurs).

⚠ Pièges fréquents à l'examen

- Oublier d'appeler super(...) alors que la classe mère n'a pas de constructeur sans paramètre.
- Mettre super(...) ailleurs qu'en première ligne du constructeur (erreur de compilation).
- Croire qu'on peut faire extends A, B (héritage multiple de classes — interdit en Java).
- Changer le type de retour ou les paramètres en pensant "redéfinir" (c'est en réalité une surcharge, pas une redéfinition).



Mini-quiz de révision — Chapitre 5

1. Que fait exactement super(nom); dans un constructeur ?

→ Il appelle le constructeur de la classe mère avec l'argument nom.

2. Combien de classes une classe Java peut-elle étendre avec extends ?

→ Une seule (pas d'héritage multiple de classes en Java).

3. Quelle est la différence entre protected et private ?

→ private = accessible seulement dans la même classe ; protected = accessible aussi dans le même package et dans les classes filles.

Chapitre 6 — Le polymorphisme

1. L'idée : une même méthode, plusieurs comportements

Le polymorphisme (« plusieurs formes ») permet à une variable de type classe mère de contenir en réalité un objet d'une classe fille, et d'appeler automatiquement la bonne version (redéfinie) de la méthode selon le type RÉEL de l'objet, et non le type déclaré de la variable.

```
public class Animal {
    public void faireDuBruit() {
        System.out.println("L'animal fait un bruit");
    }
}
public class Chien extends Animal {
    @Override
    public void faireDuBruit() { System.out.println("Le chien aboie"); }
}
public class Chat extends Animal {
    @Override
    public void faireDuBruit() { System.out.println("Le chat miaule"); }
}

public static void main(String[] args) {
    Animal[] animaux = { new Chien(), new Chat() };
    for (Animal a : animaux) {
        a.faireDuBruit(); // exécute la bonne version selon l'objet RÉEL
    }
    // Affiche : "Le chien aboie" puis "Le chat miaule"
}
```

C'est l'intérêt majeur du polymorphisme : on peut écrire du code générique (une boucle, une méthode) qui manipule le type `Animal`, sans savoir à l'avance si c'est un `Chien` ou un `Chat` — et le bon comportement s'exécute automatiquement.

2. Redéfinition (overriding) vs surcharge (overloading) : LA question piège

	Redéfinition (override)	Surcharge (overload)
Où ?	Entre classe mère et classe fille	Dans la MÊME classe
Signature	Identique (même nom, mêmes paramètres)	Même nom, paramètres DIFFÉRENTS
But	Changer le comportement hérité	Proposer plusieurs façons d'appeler une méthode
Lien au polymorphisme ?	Oui, directement	Non — c'est un mécanisme différent

```
// Surcharge (overloading) : même nom, paramètres différents, MÊME classe
public int addition(int a, int b) { return a + b; }
public double addition(double a, double b) { return a + b; }
public int addition(int a, int b, int c) { return a + b + c; }
```

3. Le transtypage (casting) entre classe mère et classe fille

```
Animal a = new Chien(); // "upcasting" : toujours possible, automatique
Chien c = (Chien) a;    // "downcasting" : il faut le préciser explicitement
```

```
if (a instanceof Chien) {      // vérifier le type réel AVANT de downcaster, pour
    éviter une erreur
    Chien c2 = (Chien) a;
}
```

instanceof vérifie si un objet est bien du type indiqué avant de tenter un transtypage vers ce type, ce qui évite une `ClassCastException` à l'exécution.

✓ Points clés du chapitre 6

- Le polymorphisme fait appeler la méthode redéfinie du type RÉEL de l'objet, pas du type déclaré.
- Redéfinition (override) = même signature, entre classe mère/fille. Surcharge (overload) = paramètres différents, même classe.
- Upcasting (fille → mère) est automatique ; downcasting (mère → fille) doit être écrit explicitement.
- instanceof permet de vérifier le type réel avant un downcasting, pour éviter une erreur à l'exécution.

⚠ Pièges fréquents à l'examen

- Confondre redéfinition et surcharge dans une question de cours (piège très fréquent).
- Downcaster sans vérifier avec instanceof → `ClassCastException` si le type ne correspond pas.
- Croire qu'une variable de type `Animal` ne peut appeler que les méthodes définies dans `Animal` (faux : elle appelle la version redéfinie, si elle existe, du VRAI objet).

🧠 Mini-quiz de révision — Chapitre 6

1. `Animal a = new Chien(); a.faireDuBruit();` — quelle version est exécutée ?

→ Celle de `Chien`, car c'est le type RÉEL de l'objet qui compte (polymorphisme).

2. Quelle est la différence entre surcharge et redéfinition ?

→ La surcharge se fait dans une même classe avec des paramètres différents ; la redéfinition se fait entre classe mère et fille avec la MÊME signature.

3. Pourquoi utiliser instanceof avant un downcasting ?

→ Pour vérifier que l'objet est bien du type ciblé, et éviter une `ClassCastException`.

Chapitre 7 — Les classes abstraites

1. Pourquoi une classe abstraite ?

Une classe abstraite représente un concept trop général pour être instancié directement. On ne crée jamais un objet « Animal » tout court dans la vraie vie — on crée toujours un chien, un chat, etc. La classe abstraite sert de modèle commun, avec certaines méthodes déjà écrites, et d'autres laissées à définir par les classes filles.

```
public abstract class Animal {  
    protected String nom;  
  
    public Animal(String nom) { this.nom = nom; }  
  
    public void dormir() {                // méthode CONCRÈTE, déjà écrite  
        System.out.println(nom + " dort.");  
    }  
  
    public abstract void faireDuBruit();    // méthode ABSTRAITE : pas de corps !  
}
```

Règles à retenir : impossible de faire `new Animal(...)` directement — une classe `abstract` sert uniquement de modèle à hériter. Toute classe fille `CONCRÈTE` (non abstraite) doit obligatoirement implémenter toutes les méthodes abstraites héritées, sinon elle ne compile pas.

```
public class Chien extends Animal {  
    public Chien(String nom) { super(nom); }  
  
    @Override  
    public void faireDuBruit() {          // obligatoire : implémenter la  
méthode abstraite  
        System.out.println(nom + " aboie.");  
    }  
}
```

2. Classe abstraite vs classe normale

	Classe normale	Classe abstraite
On peut faire <code>new ... ()</code> ?	Oui	Non, jamais directement
Peut avoir des méthodes sans corps ?	Non	Oui (méthodes <code>abstract</code>)
Peut avoir des méthodes avec corps ?	Oui	Oui (mélange possible)
But	Être utilisée directement	Servir de modèle à hériter

✓ Points clés du chapitre 7

- `abstract class` = modèle qu'on ne peut PAS instancier directement (pas de `new`).
- Une méthode `abstract` n'a pas de corps (`{ }`) : juste sa signature suivie d'un point-virgule.
- Une classe fille concrète DOIT implémenter toutes les méthodes abstraites héritées.
- Une classe abstraite peut mélanger méthodes abstraites et méthodes déjà codées (concrètes).

⚠ Pièges fréquents à l'examen

- Essayer de faire `new Animal(...)` sur une classe abstraite → erreur de compilation.
- Oublier d'implémenter une méthode abstraite dans une classe fille concrète → erreur de compilation.
- Donner un corps `{ }` à une méthode déclarée `abstract` (interdit, contradictoire).



Mini-quiz de révision — Chapitre 7

1. Peut-on écrire `Animal a = new Animal("Rex");` si `Animal` est abstraite ?

→ *Non, c'est interdit : une classe abstraite ne peut jamais être instanciée directement.*

2. Que se passe-t-il si une classe fille concrète n'implémente pas une méthode abstraite héritée ?

→ *Le code ne compile pas.*

Chapitre 8 — Les interfaces

1. C'est quoi une interface ?

Une interface est un contrat pur : elle décrit uniquement CE QUE des classes doivent savoir faire (des méthodes, en général sans aucun corps), sans dire COMMENT. Toute classe qui « implémente » une interface s'engage à écrire le corps de toutes ses méthodes.

```
public interface Volant {  
    void voler();           // pas de corps -> juste la signature  
}  
  
public interface Nageur {  
    void nager();  
}  
  
public class Canard implements Volant, Nageur {    // implémente PLUSIEURS interfaces !  
    @Override  
    public void voler() { System.out.println("Le canard vole."); }  
  
    @Override  
    public void nager() { System.out.println("Le canard nage."); }  
}
```

C'est la grande différence avec l'héritage de classe : une classe ne peut étendre (extends) qu'UNE seule classe, mais peut implémenter (implements) AUTANT d'interfaces qu'elle veut. C'est la façon dont Java compense l'absence d'héritage multiple de classes.

2. Interface vs classe abstraite : le tableau à connaître par cœur

	Interface	Classe abstraite
Mot-clé pour l'utiliser	implements	extends
Combien à la fois ?	Plusieurs possibles	Une seule
Attributs ?	Seulement des constantes (public static final)	N'importe quel attribut
Méthodes avec corps ?	Possible depuis Java 8 (méthodes default)	Oui
Constructeur ?	Non, jamais	Oui

3. Les méthodes default (depuis Java 8)

Une interface peut aussi fournir une méthode déjà codée, avec le mot-clé default, que les classes qui implémentent peuvent utiliser telle quelle ou redéfinir si besoin :

```
public interface Volant {  
    void voler();  
  
    default void decoller() {           // méthode déjà codée dans l'interface  
        System.out.println("Décollage...");  
    }  
}
```

- implements = une classe implémente une interface (contrat de méthodes).
- Une classe peut implémenter PLUSIEURS interfaces (contrairement à extends, limité à une classe).
- Les attributs d'une interface sont implicitement public static final (des constantes).
- Une méthode default a un corps directement dans l'interface, depuis Java 8.
- Une interface n'a jamais de constructeur (on ne peut jamais faire new SurUneInterface()).

⚠ Pièges fréquents à l'examen

- Écrire extends au lieu de implements pour une interface.
- Oublier d'implémenter une méthode de l'interface dans la classe → erreur de compilation.
- Croire qu'on peut créer un objet directement à partir d'une interface (impossible, comme pour une classe abstraite).

🧠 Mini-quiz de révision — Chapitre 8

1. Combien d'interfaces une classe Java peut-elle implémenter ?

→ *Autant qu'elle veut, il n'y a pas de limite.*

2. Quel mot-clé utilise-t-on pour implémenter une interface ?

→ *implements*

3. Une interface peut-elle avoir un constructeur ?

→ *Non, jamais.*

Chapitre 9 — La gestion des exceptions

1. C'est quoi une exception ?

Une exception est une erreur qui survient PENDANT l'exécution du programme (division par zéro, accès à un indice invalide, fichier introuvable...). Sans gestion, le programme s'arrête brutalement. Le mécanisme try/catch permet de « rattraper » l'erreur et de continuer proprement.

```
try {
    int[] t = {1, 2, 3};
    System.out.println(t[5]);          // erreur : indice invalide
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Indice invalide : " + e.getMessage());
} finally {
    System.out.println("Ce bloc s'exécute TOUJOURS, erreur ou pas.");
}
```

Bloc	Rôle
try	Contient le code risqué à surveiller
catch	Rattrape et traite une exception si elle survient
finally	S'exécute toujours à la fin, qu'il y ait eu erreur ou non (souvent pour fermer un fichier, une connexion...)

2. Exceptions vérifiées vs non vérifiées (checked / unchecked)

	Checked (vérifiée)	Unchecked (non vérifiée)
Exemple	IOException	NullPointerException, ArithmeticException
Le compilateur oblige à la gérer ?	Oui (try/catch ou throws obligatoire)	Non, c'est facultatif
Cause typique	Problème externe imprévisible (fichier, réseau)	Erreur de programmation

Exceptions classiques à connaître : ArithmeticException (division par zéro), NullPointerException (utiliser un objet null), ArrayIndexOutOfBoundsException (indice de tableau invalide), NumberFormatException (conversion texte → nombre impossible), ClassCastException (transtypage invalide, voir chapitre 6).

3. Créer et lancer sa propre exception

```
public class AgeInvalideException extends Exception {
    public AgeInvalideException(String message) {
        super(message);
    }
}

public void setAge(int age) throws AgeInvalideException {
    if (age < 0) {
        throw new AgeInvalideException("L'âge ne peut pas être négatif");
    }
    this.age = age;
}
```

}

throws (dans la signature de la méthode) annonce qu'elle PEUT lancer cette exception. throw (dans le corps) déclenche RÉELLEMENT l'exception à cet instant précis. Ne pas confondre les deux mots, très proches à l'écrit.

✓ Points clés du chapitre 9

- try = code surveillé ; catch = traitement de l'erreur ; finally = s'exécute toujours.
- throws (dans la signature) annonce une exception possible ; throw (dans le corps) la déclenche réellement.
- Checked exceptions : gestion obligatoire par le compilateur (ex: IOException). Unchecked : facultative (ex: NullPointerException).
- Une exception personnalisée hérite généralement de Exception (checked) ou RuntimeException (unchecked).

⚠ Pièges fréquents à l'examen

- Confondre throw et throws.
- Oublier de gérer (try/catch ou throws) une exception checked → erreur de compilation.
- Mettre le catch le plus spécifique APRÈS un catch plus général (Exception) : le code devient inatteignable et ne compile pas.

Mini-quiz de révision — Chapitre 9

1. Quelle est la différence entre throw et throws ?

→ throw déclenche réellement une exception dans le corps du code ; throws l'annonce dans la signature d'une méthode.

2. Quel bloc s'exécute toujours, même en cas d'erreur ?

→ finally

3. Citer une exception unchecked liée à un objet non initialisé.

→ NullPointerException

Chapitre 10 — Les collections

1. Pourquoi les collections plutôt que les tableaux ?

Un tableau a une taille FIXE, choisie une fois pour toutes. Les collections (package `java.util`) sont des structures qui peuvent grandir ou rétrécir librement pendant l'exécution — beaucoup plus pratiques dans la majorité des cas réels.

2. ArrayList : le tableau qui grandit tout seul

```
import java.util.ArrayList;
import java.util.List;

List<String> noms = new ArrayList<>(); // <String> = ne contient QUE des String
(générique, voir chap. 11)

noms.add("Alice");
noms.add("Bob");
noms.add("Chloé");

System.out.println(noms.get(1)); // "Bob" (accès par indice, comme un tableau)
noms.remove("Bob"); // enlève l'élément "Bob"
System.out.println(noms.size()); // 2 (PAS .length !)

for (String n : noms) { // for-each, comme pour un tableau
    System.out.println(n);
}
```

Piège d'examen classique : sur une `ArrayList`, on utilise `.size()` (méthode, avec parenthèses), jamais `.length` ni `.length()`.

3. HashMap : stocker des paires clé → valeur

Une `HashMap` associe chaque clé unique à une valeur, comme un dictionnaire ou un carnet d'adresses (nom → numéro de téléphone).

```
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> notes = new HashMap<>();
notes.put("Alice", 18);
notes.put("Bob", 12);

System.out.println(notes.get("Alice")); // 18
System.out.println(notes.containsKey("Bob")); // true
notes.remove("Bob");

for (Map.Entry<String, Integer> entree : notes.entrySet()) {
    System.out.println(entree.getKey() + " -> " + entree.getValue());
}
```

4. HashSet : une collection sans doublons

```
import java.util.HashSet;
import java.util.Set;

Set<String> villes = new HashSet<>();
villes.add("Marrakech");
villes.add("Rabat");
villes.add("Marrakech"); // ignoré : déjà présent, pas de doublon
```

```
System.out.println(villes.size()); // 2, pas 3
```

HashSet ne garantit pas d'ordre de parcours particulier et refuse automatiquement les doublons — utile quand on veut juste savoir « quels éléments existent », sans se soucier de l'ordre ni des répétitions.

5. Comparatif rapide des trois collections principales

Collection	Contient	Doublons ?	Accès typique
ArrayList	Une liste ordonnée d'éléments	Oui, autorisés	Par indice : <code>get(i)</code>
HashSet	Un ensemble d'éléments	Non, interdits	Pas d'indice, juste <code>contains()</code>
HashMap	Des paires clé → valeur	Clés uniques, valeurs libres	Par clé : <code>get(clé)</code>

✓ Points clés du chapitre 10

- ArrayList = liste à taille variable, accès par indice avec `get(i)`, taille avec `.size()`.
- HashMap = associe une clé unique à une valeur ; `put()` pour ajouter, `get()` pour lire.
- HashSet = ensemble sans doublons.
- Toujours importer depuis `java.util` (ArrayList, HashMap, HashSet, List, Map, Set).

⚠ Pièges fréquents à l'examen

- Utiliser `.length` ou `.length()` sur une ArrayList au lieu de `.size()`.
- Oublier que HashSet et HashMap ne garantissent pas d'ordre de parcours précis.
- Confondre List (interface) et ArrayList (classe concrète qui l'implémente).

🧠 Mini-quiz de révision — Chapitre 10

1. Comment obtenir le nombre d'éléments d'une ArrayList nommée `liste` ?

→ `liste.size()`

2. Qu'arrive-t-il si on ajoute deux fois le même élément dans un HashSet ?

→ Le doublon est ignoré : l'élément n'apparaît qu'une seule fois.

3. Quelle méthode utilise-t-on pour ajouter une paire clé/valeur dans une HashMap ?

→ `put(clé, valeur)`

Chapitre 11 — Les génériques (generics)

1. Le problème que les génériques résolvent

Avant les génériques, une collection pouvait contenir n'importe quel type d'objet mélangé, ce qui provoquait des erreurs seulement à l'EXÉCUTION (par exemple, en essayant de traiter un texte comme un nombre). Les génériques permettent de préciser, dès l'écriture du code, quel type une classe ou une méthode doit manipuler — et l'erreur est alors détectée à la COMPILATION, bien plus tôt.

```
List<String> noms = new ArrayList<>();
noms.add("Alice");
noms.add(42);           // ERREUR DE COMPILATION : 42 n'est pas une String -> détecté
                        immédiatement
```

2. Écrire sa propre classe générique

On utilise une lettre entre chevrons (par convention T pour Type, E pour Element, K/V pour Key/Value) comme un type "à remplir plus tard".

```
public class Boite<T> {
    private T contenu;

    public void ranger(T objet) { this.contenu = objet; }
    public T recuperer() { return contenu; }
}

Boite<String> boiteTexte = new Boite<>();
boiteTexte.ranger("Bonjour");
String s = boiteTexte.recuperer();    // pas besoin de transtypage (cast) !

Boite<Integer> boiteNombre = new Boite<>();
boiteNombre.ranger(42);
```

Avantage concret : plus besoin de faire un transtypage (cast) manuel à la récupération, et toute tentative de ranger un type incompatible est bloquée dès la compilation.

3. Méthode générique

```
public static <T> void afficherTableau(T[] tableau) {
    for (T element : tableau) {
        System.out.println(element);
    }
}
// Utilisable avec String[], Integer[], ou n'importe quel autre tableau d'objets
```

✓ Points clés du chapitre 11

- Les génériques précisent un type dès l'écriture du code, avec des chevrons < >.
- Ils déplacent la détection d'erreur de type de l'EXÉCUTION vers la COMPILATION.
- T, E, K, V sont des noms de convention pour un type générique (pas des mots-clés obligatoires).
- Avec les génériques, plus besoin de transtypage (cast) manuel à la récupération d'un élément.

⚠ Pièges fréquents à l'examen

- Oublier les chevrons <> à la création : `new ArrayList()` au lieu de `new ArrayList<String>()`.

- Utiliser un type primitif (int) au lieu de sa version objet (Integer) dans un générique — les génériques n'acceptent que des types objets.

Mini-quiz de révision — Chapitre 11

1. Peut-on écrire List<int> ?

→ Non : les génériques n'acceptent que des types objets, donc List<Integer>.

2. Quel est l'avantage principal des génériques ?

→ Détecter les erreurs de type dès la compilation plutôt qu'à l'exécution, et éviter les transtypages manuels.

Chapitre 12 — Les énumérations (enum)

1. À quoi sert un enum ?

Un enum définit un ensemble FIXE et connu à l'avance de valeurs possibles — par exemple les jours de la semaine, les couleurs d'un jeu de cartes, ou le statut d'une commande. C'est plus sûr qu'utiliser des String ou des int « à la main », car le compilateur interdit toute valeur qui n'est pas dans la liste.

```
public enum Jour {  
    LUNDI, MARDI, MERCREDI, JEUDI, VENDREDI, SAMEDI, DIMANCHE  
}  
  
Jour aujourdHui = Jour.MERCREDI;  
  
switch (aujourdHui) {  
    case LUNDI -> System.out.println("Début de semaine");  
    case SAMEDI, DIMANCHE -> System.out.println("Week-end !");  
    default -> System.out.println("Jour de semaine");  
}
```

2. Un enum peut avoir des attributs et méthodes

```
public enum Planete {  
    MERCURE(3.7), TERRE(9.8), JUPITER(24.8);    // chaque constante appelle le  
    constructeur  
  
    private final double gravite;  
  
    Planete(double gravite) {                    // constructeur toujours private  
(implicite)  
        this.gravite = gravite;  
    }  
  
    public double getGravite() {  
        return gravite;  
    }  
}  
  
System.out.println(Planete.TERRE.getGravite());    // 9.8
```

✓ Points clés du chapitre 12

- enum définit un ensemble fixe de valeurs connues à l'avance.
- Le compilateur interdit toute valeur qui n'est pas dans la liste : plus sûr qu'une String libre.
- Un enum peut avoir des attributs, un constructeur (toujours implicitement private) et des méthodes.
- On accède à une constante avec NomEnum.CONSTANTE (comme un attribut static).

⚠ Pièges fréquents à l'examen

- Essayer de faire new Jour() directement — impossible, les constantes sont déjà créées automatiquement par Java.
- Oublier que les noms de constantes enum sont conventionnellement tout en MAJUSCULES.

Mini-quiz de révision — Chapitre 12

1. Pourquoi préférer un enum à une simple String pour représenter un jour de la semaine ?

→ *Parce que le compilateur empêche d'utiliser une valeur invalide, contrairement à une String libre.*

2. Peut-on créer une nouvelle constante d'un enum avec new ?

→ *Non, les constantes sont fixées à l'avance dans la définition de l'enum.*

Chapitre 13 — Classes internes et classes anonymes

1. Classe interne (inner class)

Une classe interne est définie À L'INTÉRIEUR d'une autre classe. Elle est utile quand une classe n'a de sens que liée à une autre (par exemple, un Moteur n'existe que dans une Voiture).

```
public class Voiture {
    private String marque;

    public Voiture(String marque) { this.marque = marque; }

    public class Moteur {                // classe interne : liée à un objet Voiture
précis
        public void demarrer() {
            System.out.println("Le moteur de la " + marque + " démarre.");    // accès
direct à l'attribut de Voiture
        }
    }
}

Voiture v = new Voiture("Renault");
Voiture.Moteur m = v.new Moteur();    // syntaxe particulière pour créer une classe
interne
m.demarrer();
```

2. Classe anonyme : une classe sans nom, créée sur place

Une classe anonyme est utile quand on a besoin d'un objet une seule fois, souvent pour implémenter rapidement une interface, sans créer un fichier séparé pour une classe qui ne sera utilisée qu'à cet endroit.

```
public interface Operation {
    int calculer(int a, int b);
}

Operation addition = new Operation() {    // classe anonyme : implémente Operation
sur place
    @Override
    public int calculer(int a, int b) {
        return a + b;
    }
};

System.out.println(addition.calculer(3, 4));    // 7
```

Remarque culture générale : depuis Java 8, pour une interface avec une seule méthode (« interface fonctionnelle »), on peut souvent remplacer une classe anonyme par une expression lambda, plus courte :
Operation addition = (a, b) -> a + b;

✓ Points clés du chapitre 13

- Une classe interne est définie dans une autre classe et peut accéder directement à ses attributs.
- Créer une instance d'une classe interne non-static se fait via objetExterne.new ClasseInterne().
- Une classe anonyme définit et instancie une classe en une seule fois, sans lui donner de nom.
- Une classe anonyme est très utilisée pour implémenter rapidement une interface à un seul usage.

⚠ Pièges fréquents à l'examen

- Oublier la syntaxe particulière `objetExterne.new ClasseInterne()` pour une classe interne non-static.
- Croire qu'une classe anonyme peut avoir un nom réutilisable ailleurs — par définition, elle n'en a pas.

🧠 Mini-quiz de révision — Chapitre 13

1. Comment crée-t-on une instance d'une classe interne `Moteur` définie dans `Voiture`, à partir d'un objet `v` de type `Voiture` ?

→ `v.new Moteur();`

2. Dans quel cas utilise-t-on typiquement une classe anonyme ?

→ *Pour implémenter rapidement une interface à un seul usage, sans créer une classe séparée.*

Fiche de synthèse rapide (veille d'examen)

Chapitre 1 — Bases

- JDK = JRE + outils (javac, jar, javadoc).
- javac compile → java exécute (sans .class).
- int / int = division entière ; utiliser double pour garder les décimales.
- for = nombre connu ; while = testé avant ; do...while = testé après.

Chapitre 2 — Tableaux

- Indices de 0 à length - 1 ; taille fixe après création.
- tableau.length SANS parenthèses.
- Filtrage = compter puis allouer puis remplir.
- Tri à bulles = échanges de voisins ; tri par sélection = un seul échange après recherche du minimum.

Chapitre 3 — Classes et objets

- Classe = plan ; objet = instance créée avec new.
- this.attribut = attribut ; pour éviter l'ambiguïté avec le paramètre.
- private + get/set = encapsulation.
- static = partagé par tous les objets, appelé via le nom de la classe.
- equals() compare le contenu ; == compare la mémoire.

Chapitre 4 — Chaînes de caractères

- String est immuable : toujours réaffecter le résultat d'une méthode.
- Comparer deux String avec .equals(), jamais avec ==.
- StringBuffer = mutable + synchronisé, pratique pour de nombreuses modifications.
- StringTokenizer découpe une chaîne en mots selon des délimiteurs.

Chapitre 5 — Héritage

- extends = héritage, une seule classe mère possible ; super(...) appelle son constructeur.
- protected = visible dans le package + les classes filles.

Chapitre 6 — Polymorphisme

- La méthode exécutée dépend du type RÉEL de l'objet, pas du type déclaré.
- Redéfinition (même signature, mère/fille) ≠ surcharge (paramètres différents, même classe).
- instanceof avant un downcasting pour éviter une ClassCastException.

Chapitre 7 — Classes abstraites

- abstract class : jamais de new direct ; peut mélanger méthodes abstraites et concrètes.
- Une classe fille concrète doit implémenter toutes les méthodes abstraites héritées.

Chapitre 8 — Interfaces

- implements, plusieurs interfaces possibles (contrairement à extends).
- Attributs d'interface = constantes (public static final) ; méthodes default = déjà codées.

Chapitre 9 — Exceptions

- try = code surveillé ; catch = traitement ; finally = toujours exécuté.
- throw déclenche réellement ; throws l'annonce dans la signature.

Chapitre 10 — Collections

- ArrayList (liste), HashSet (sans doublons), HashMap (clé → valeur).
- Toujours .size(), jamais .length/.length() sur une collection.

Chapitre 11 — Génériques

- <T> précise un type dès l'écriture : erreurs détectées à la compilation, pas de cast manuel.

Chapitre 12 — Enum

- Ensemble fixe de constantes connues à l'avance ; peut avoir attributs/méthodes.

Chapitre 13 — Classes internes et anonymes

- Classe interne : définie dans une autre classe, accès direct à ses attributs (objet.new Classe()).
- Classe anonyme : classe sans nom, créée et instanciée en une fois, souvent pour une interface.